

SYSTEM DESIGN REVIEW / 面试复盘手册

AgentLoom

多智能体 workflow 编排平台

一个把可视化 DAG 编排、独立 Agent 运行时、Firecracker 微虚拟机沙箱、签名插件生态与企业级治理面收拢进同一套多租户契约的平台。本手册按“设计意图 → 结构 → 取舍 → 追问”组织，用于快速重建心智模型。

代码规模

686k 行

服务端模块

41 个

数据表

74 张

提交

1,031 次

HOW TO READ · 00

这份文档要解决的 不是“有什么功能”

面试官不会问“你有几个模块”，只会问“这里为什么这么做、不这么做会怎样、边界在哪”。所以本手册的每一节都固定回答三件事：**结构是什么、为什么这样切、失败时会怎样**。

01

结构

用图和表给出模块、数据流、协议信封。落到具体文件路径和表名，方便当场翻代码。

02

取舍

每个设计都记录被放弃的替代方案，以及放弃的代价。这是面试里最值钱的部分。

03

失效

fail-closed 点、竞态防护、幂等键、降级路径。答不出失效模式等于没设计过。

■ 三条贯穿全局的主线

- **快照优先** — workflow、Agent、插件、生成应用全部“草稿可改、发布不可变”。执行默认读快照；唯一例外是编辑器里的 Run（`launchSource='web-studio'`）会读当前草稿，见 B3。
- **租户是第一等公民** — 每个请求包在一个设置了 `app.current_tenant` 的数据库事务里，PostgreSQL RLS 兜底，而不是靠代码里记得加 `where tenant_id`。
- **不可信代码只在一个地方跑** — Agent 生成的代码进 Firecracker 微虚拟机，插件进 Extism WASM。业务进程既没有 KVM 也没有 Docker socket。

SELF-HONESTY

文档末尾有一章「**技术债与漂移**」，逐条列出当前实现与设计意图不一致的地方。面试时主动讲这些，比被追问出来强得多。

CONVENTION

正文中 等宽字体 一律是真实存在的文件、符号、表名或接口路径；未经实测的推断会显式标注为「推断」。

CONTENTS · 目录

六个部分

A	全景	
A1	项目定位与产品形态	06
A2	系统全景架构	07
A3	仓库结构与包边界	08
A4	数据模型全景 · 74 张表	09
B	执行内核	
B1	服务端骨架与全局管线	11
B2	多租户：RLS + 事务上下文	12
B3	定义、版本与发布语义	13
B4	runWorkflow 全链路	14
B5	NodeScheduler：DAG 调度	15
B6	复合节点：循环与分支	16
B7	Checkpoint 与断点续跑	17
B8	实时协议与事件回放	18
B9	背压、队列与触发器	19
C	智能体	
C1	Agent 与 Workflow 的并行关系	21
C2	版本漂移与对话刷新	22
C3	双运行态与适配器分层	23
C4	Skill 注入体系	24
C5	Memory 图谱	25
C6	自进化与审批边界	26

D	隔离与生态	
D1	Firecracker 沙箱拓扑	28
D2	网络身份边界	29
D3	会话生命周期与工作区租约	30
D4	ACP 网关	31
D5	回调中继与 SSRF 防护	32
D6	插件包与规范化签名	33
D7	WASM 执行与分成结算	34
D8	市场、分享与来源分类	35
D9	Generated App 八道门禁	36
E	平台面	
E1	Studio 架构与状态分层	38
E2	画布节点与端口契约	39
E3	Rust/WASM 类型引擎	40
E4	认证与端到端加密	41
E5	审计：append-only 双表	42
E6	配额、限流与监控	43
E7	知识库 RAG 与 MCP	44
E8	移动端	45
E9	部署拓扑与备份恢复	46
F	复盘	
F1	技术债与契约漂移	48
F2	面试问答速查 · 架构	49
F3	面试问答速查 · 并发与一致性	50
F4	面试问答速查 · 安全	51
F5	一分钟自我介绍脚本	52

AT A GLANCE · 数字速览

规模

数字来自本地工作树实测（`find + wc -l`，排除 `node_modules`），非估算。

686k

总代码行数

1,031

GIT 提交 · 08-13

5.2

开发月数

11

子包

■ 各包体量

包	技术栈	文件	行数
agentloom-server	NestJS 11 / Fastify 5	1,805	399,862
agentloom-studio	React 19 / Vite 7	1,089	198,148
agentloom_mobile	Flutter 3.41	305	73,777
firecracker-runtime	Go	51	8,780
type-engine	Rust 2024 / WASM	26	2,005
plugin-cli	TypeScript / Commander	18	1,961
plugin-sdk	TypeScript / Zod 3	27	1,786

■ 测试与门禁

334

*.SPEC.TS

299

*.TEST.TS(X)

92

*_TEST.DART

80%

SERVER 覆盖率门槛

TIMELINE

首次提交 `f1d766b` · 2026-03-07

最近提交 · 2026-08-13

约 5 个月、平均每天 6~7 次提交的高强度 AI 辅助开发。

SERVER

41

业务模块目录

POSTGRES

74

Drizzle pgTable

SOCKET.IO

5

实时 namespace

BULLMQ

10+

队列与调度器

PART A



全景

先建立坐标系：这个平台在解决什么问题、由哪些运行单元组成、代码怎么切包、数据怎么分域。后面所有细节都挂在这张骨架上。

A 1

项目定位与产品形态

A 2

系统全景架构

A 3

仓库结构与包边界

A 4

数据模型全景

A1 · POSITIONING

把「编排」和「智能体」当成两个顶层概念

市面上大多数同类产品只选一边：要么是 DAG 编排器（节点里塞一个 LLM 调用），要么是 Agent 框架（用对话驱动一切）。AgentLoom 把两者做成**并列的顶层实体**，各自拥有定义、版本、执行记录与恢复边界，再用一个适配器把 Agent 嵌进 workflow 节点。

CONCEPT 01

Workflow · 确定性编排

ReactFlow DAG，节点类型 30+，含条件、循环、迭代、合并、插件、沙箱。由 NodeSchedulerService 拓扑调度，状态落 workflow_executions / execution_steps，可断点续跑。

适合：流程固定、要审计、要重放。

CONCEPT 02

Agent · 自主对话

独立的 Agent 定义 / 版本 / 对话 / 消息体系，带模型、工具、知识库、记忆、Skill、子 Agent。运行在 Firecracker 沙箱或进程内 pi-agent-core。

适合：路径不确定、需要探索和写代码。

WorkflowAgentAdapter (src/modules/execution/workflow-agent-adapter.ts) 按 agentVersionId 或当前 publishedVersionId 取 Agent 快照，编译成统一 runtime session 以 mode:'workflow' 运行，输出经 EventBridge 转成 step / output / tool 事件。**Agent 逻辑只有一份**，工作流侧不复制。

■ 产品面上的六条主线

- 可视化编排与执行监控
- 独立 Agent 对话与自进化
- 自然语言生成应用 (Generated App)
- 签名插件生态与市场分成
- 知识库 RAG、Memory 图谱、MCP 工具
- 企业治理：审计、配额、监控、私有部署

■ 为什么不把 Agent 做成一种节点类型

因为**生命周期和故障恢复边界不同**。工作流执行是一次性的、有确定终点的 DAG；Agent 对话是长期存在、可以跨会话继续、需要独立版本发布和权限策略的实体。如果 Agent 只是节点，就没法在工作流之外单独复用它，也没法给它单独的版本审计线。

反过来，如果一切都是 Agent，就失去了 DAG 的确定性、可视化和逐步重放能力——这正是企业客户最想要的部分。

A2 · TOPOLOGY

运行单元与信任域

整套系统在部署上分成四类进程，划分依据不是「功能」，而是**持有什么权限**。这是理解整个架构最快的入口。

Studio (React) / Mobile (Flutter) — 浏览器与设备内，只持有用户 JWT 与本地私钥	CLIENT
server — 公网 REST <code>/api/v1</code> + Socket.IO 入口。无 Docker socket，无 KVM，无 NET_ADMIN	PUBLIC EDGE
worker — 同一镜像同一入口，只处理队列与内部回调中继，不对公网暴露	INTERNAL
firecracker-runtime (Go) — 唯一 privileged 单例：/dev/kvm、TUN、cgroup、CNI、nftables、jailer	PRIVILEGED
microVM guest — pi-coding-agent + Fastify + agentloom-guestd，被认为 完全不可信	UNTRUSTED

■ 数据面依赖

PostgreSQL SUPABASE / DRIZZLE 租户数据 · RLS · 74 表	Redis BULLMQ 队列 · 调度 · 限 流 · Socket adapter	Qdrant VECTOR 知识库向量检索	MinIO S3 文档 · 工作区 · 插件 · Skill
--	--	------------------------------------	--

■ 五个实时 namespace

<code>/execution</code> · <code>/agent-conversation</code> · <code>/memory</code> · <code>/knowledge</code> · <code>/notification</code>
--

■ 一次 workflow 执行的数据流

- 01 Studio POST `/workflow-definitions/:id/run`
- 02 治理准入判定 → 写 `workflow_executions`
(pending)
- 03 事务提交后才入队 `workflow-execution`
- 04 Worker 初始化 `execution_steps`，交给
NodeScheduler
- 05 逐层调度，Agent 节点走 `agent-task` 队列、插
件走 `plugin-execution`
- 06 事件经 EventBridge 编号后广播到 `/execution`
- 07 前端按 `lastEventId` 增量消费，断线可回放

■ 为什么权限要按进程切

Agent 会执行模型生成的任意代码。如果业务进程同时持有 KVM 或 Docker socket，一旦应用层出现 RCE 或 SSRF，攻击者就直接拿到宿主。

把「能创建虚拟机」这件事收敛到一个单例、只说 mTLS、只接受白名单调用的 Go 进程后，业务侧的漏洞最多影响业务数据，不会升级为宿主逃逸。

TRADE-OFF

代价是 runtime manager 成为单点：Helm 里它是 `replicas=1` 的 StatefulSet，绑 RWO 卷、绑节点。HA 需要另做状态复制，当前明确不做。

A3 · REPOSITORY

十一个包，没有 workspace

这是一个**非标准 monorepo**：根目录没有 `pnpm-workspace.yaml`，每个包自带 lockfile 和命令。好处是包之间没有隐式提升依赖、可以各自锁版本（例如 SDK 停在 Zod 3 而服务端用 Zod 4）；代价是没有统一的一键构建，跨包重构要手动同步。

包	技术栈	职责与边界
agentloom-server	NestJS 11 · Fastify 5 · Drizzle	唯一权威：认证、租户、定义、执行编排、插件注册、门禁、证据、审计、治理
agentloom-studio	React 19 · Vite 7 · React Flow	画布编辑、执行监控、Agent 对话、全部管理页；只是契约的消费方
agentloom_mobile	Flutter 3.41 · Riverpod	观测与操作（启动、监控、对话、轻资源管理）， 不做编辑器
firecracker-runtime	Go	microVM 生命周期、CNI/nft、jailer、mutable disk、guest mTLS 代理
type-engine	Rust 2024 · wasm-bindgen	端口 schema 兼容性判定，编译成 WASM 在浏览器 Worker 内跑
plugin-sdk	TypeScript · Zod 3 · tsup	插件类型、校验、RSA-PSS 规范化签名工具，双 ESM/CJS 输出
plugin-cli	Commander · archiver	create / dev / build / keys generate / publish
agentloom-deploy	Compose · Helm · Shell	私有化部署资产、备份恢复、Firecracker 产物构建链
agentloom-docs	VitePress 2	中英 locale 文档站，构建前同步 OpenAPI spec
plugin-template	SDK 示例插件	text-to-uppercase，作为脚手架产物的参考实现
agentloom-user-docs	VitePress	面向终端用户的文档站，与开发者文档分开

KNOWN COST

没有共享类型包。端口类型、事件信封等契约靠约定手工镜像在 Rust / Studio / Server / SDK 四处，已经产生实际漂移（见 F1）。如果重做，第一件事就是抽一个 `@agentloom/contracts`。

NO CI

项目只有本地开发环境，没有 CI/CD。质量靠 Vitest 80% 覆盖率门槛 + Testcontainers E2E 本地跑，这是个明确的短板。

A4 · DATA MODEL

74 张表，八个域

统计口径：`src/database/schema/*.ts` 中的 `pgTable()` 定义去重计数。

编排域 · 12

`workflow_definitions` `workflow_versions`
`workflow_executions` `execution_steps`
`workflow_triggers` `workflow_trigger_history`
`workflow_templates` `workflow_shares`
`reusable_blocks` `intervention_policies`
`execution_governance_controls`
`tool_definitions`

智能体域 · 10

`agent_definitions` `agent_versions`
`agent_conversations` `agent_messages`
`agent_execution_records` `agent_shares`
`acp_conversation_sessions` `skills`
`optimization_suggestions`
`organization_autonomy_policies`

记忆域 · 7

`agent_memory_instances` `memory_nodes`
`memory_edges` `memory_versions` `memory_paths`
`memory_sessions` `memory_glossary_keywords`

沙箱与工作区 · 5

`sandbox_sessions` `sandbox_logs`
`sandbox_runtime_migrations`
`workspace_snapshots`
`workspace_runtime_leases`

生态与计费 · 11

`plugins` `plugin_developer_keys`
`plugin_usage_records` `plugin_earnings`
`marketplace_listings` `marketplace_reviews`
`generated_apps` 及其 `gate_runs` /
`generation_runs` / `repair_attempts` /
`submissions`

知识与模型 · 9

`knowledge_bases` `knowledge_nodes` `documents`
`llm_providers` `llm_model_configs`
`provider_health_status` `router_models`
`routing_decisions` `routing_benchmarks`

治理与安全 · 9

`audit_logs` `audit_log_archives`
`evidence_records` `evidence_export_jobs`
`tenant_quotas` `tenant_encryption_keys`
`private_deployment_settings`
`platform_api_tokens` `revoked_tokens`

身份与其他 · 11

`users` `organizations` `organization_members`
`organization_invitations` `user_preferences`
`notifications` `notification_preferences`
`device_tokens` `api_keys` `mcp_server_configs`
`resource_source_records`

PATTERN

几乎所有业务表都带 `tenant_id` 并启用 `FORCE ROW LEVEL SECURITY`。子表（如 `execution_steps`）不重复存租户，而是用 `createJoinTenantPolicies` 生成 `EXISTS(父表 WHERE tenant_id = get_tenant_id())` 的策略——少一列冗余，多一次索引 join，换取「租户列不会写错」的强保证。

PART B

B

执行内核

从 HTTP 请求到 DAG 节点跑完，中间隔着租户事务、发布快照、准入判定、队列、拓扑排序、复合节点状态机、检查点和事件回放。这一部分是整个系统的核心，也是面试最容易深挖的地方。

B1-B3

骨架 · 多租户 · 版本语义

B4-B7

调度 · 复合节点 · 检查点

B8-B9

实时协议 · 背压 · 触发器

B1 · BOOTSTRAP

全局管线的顺序就是安全模型

src/main.ts 用 FastifyAdapter({ rawBody: true }) 起 Nest (rawBody 是给 webhook HMAC 验签用的), 全局前缀 /api/v1, multipart 上限 50MB, RedisIoAdapter 让 Socket.IO 跨进程广播 (失败降级单实例), 再挂 AllExceptionsFilter + ZodValidationPipe + Swagger。

01 TenantMiddleware 无 X-API-Key 时, 从 JWT payload 不验签提取 tenantId, 只为尽早拿到租户	02 Throttler 按租户动态限流, 读 tenant_quotas.apiRateLimitPerMinute	03 AuthGuard JWT 优先验签 (HS256 / aud=authenticated / 黑名单 / MFA), 无 Bearer 才回落 X-API-Key	04 TenantGuard 校验用户确实属于该租户	05 RolesGuard owner / admin / creator / operator / viewer
--	--	--	---	--

SUBTLE

中间件里的 JWT 解析**不验签**, 这看起来像漏洞, 其实是分工: 它只负责「提前知道租户是谁」, 真正的信任建立在 AuthGuard。而事务拦截器依赖的是 request.user.tenantId ——也就是**验签之后**的值, 所以伪造 JWT 拿不到别的租户事务。这是个典型的「看起来危险、实际分层正确」的追问点。

■ 核心队列

队列	要点
workflow-execution	attempts=1, 不自动重试整条执行
agent-task	并发 10, attempts=4, 指数退避 2s
plugin-execution	由调度器注入, WASM 执行
trigger-scheduler	cron repeat + timezone
notification	attempts=3, 扇出三通道
audit-log-retention	单例 scheduler, 归档冷数据
earnings-settlement	插件收益周期结算

■ 为什么整条执行不自动重试

workflow节点会调用 LLM、写文件、发 HTTP 请求, **绝大多数副作用不幂等**。整条重跑等于重复扣费、重复下单。所以 workflow-execution 只跑一次, 失败后由用户显式 POST /executions/:id/resume, 并且 resume 会跳过已 completed 的步骤。

相反 agent-task 允许 4 次退避重试, 因为它的失败绝大多数是模型侧的瞬时错误 (限流、超时), 而单个 Agent 回合内部有自己的检查点。

INTERVIEW

「重试策略怎么定的」——按**副作用幂等性**分层, 不是按重要性分层。

B2 · MULTI-TENANCY

不靠「记得加 where」

租户隔离如果依赖每个查询都手写 `where tenant_id = ?`，那它的正确性上限就等于代码审查的严格程度。这里改成数据库强制。

■ 三段式

- 01 **迁移期建立策略。** `0005_lazy_tomorrow_man.sql` 定义 `get_tenant_id()` = `NULLIF(current_setting('app.current_tenant', true), '')::uuid`，为每张业务表 `ENABLE + FORCE ROW LEVEL SECURITY`，并按 `createDirectTenantPolicies / createAppendOnlyTenantPolicies` 生成策略。
- 02 **请求期开事务。**全局 `TenantTransactionInterceptor` 把整个 handler 包进 `runInTenantTransaction`：先 `SET LOCAL ROLE authenticated`，再 `set_config('app.current_tenant', tenantId, true)`（第三参 `true` = 事务级，事务结束自动失效）。
- 03 **取连接。**事务句柄放进 `AsyncLocalStorage`；所有 `service` 通过 `getTenantDb()` 取，拿到的永远是带租户上下文的那条连接，而不是连接池里的裸连接。

WHY FORCE

普通 `ENABLE RLS` 对表 owner 不生效。
`FORCE` 让策略对所有人生效，包括迁移用的高权限角色。配合 `SET LOCAL ROLE authenticated`，应用即使拿到超级用户连接也会被降权。

SIDE EFFECT

响应在**事务提交后**才发出。这就要求所有「提交后才能做」的副作用（比如把执行任务丢进 BullMQ）注册成 `afterCommit` hook，否则 worker 可能读到还没提交的行。

EXCEPTION

`platform_api_tokens` 没有 RLS——因为验证 API Key 时还不知道租户是谁，属于「先有鸡还是先有蛋」。这张表靠 service 层按 `token_hash` 精确查询兜底。

DIRECT

主表： `tenant_id = get_tenant_id()`，四条策略 (S/I/U/D)

JOIN

子表： `EXISTS(SELECT 1 FROM 父表 WHERE id = ... AND tenant_id = get_tenant_id())`

APPEND-ONLY

审计表：只授 `SELECT + INSERT`，数据库层面拒绝 `UPDATE / DELETE`

B3 · VERSIONING

草稿号 ≠ 版本号

这里有一个很容易被问住的点：`workflow_definitions.version` 和用户看到的「v3」**不是同一个东西**。

`workflow_definitions.version`

草稿修订号，兼作**乐观并发控制（OCC）**版本。每次自动保存 +1。前端 2 秒防抖 PATCH，服务端用它检测「你编辑的是不是最新的图」。

它会一路涨到几百，跟发布无关。

`snapshot.metadata.releaseNumber`

用户可见的发布号，**只在快照首次发布时分配**（取历史最大值 +1）。API 以 `publishedReleaseNumber` 暴露。重新发布一个历史版本会沿用它原来的号，不会造成号段跳跃。

■ 发布链路

DRAFT 编辑草稿 <code>nodes/edges/viewport</code> 直接存在定义表，version++ 作 OCC	SNAPSHOT <code>createVersion</code> 插入 <code>workflow_versions</code> ， <code>versionNumber</code> = max+1， 不发布	VALIDATE <code>publish</code> 前校验 图合法性、Agent 绑定存在、 no_sandbox 节点不得引用 stdio MCP	PUBLISH 原子切换 清旧 <code>publishedAt</code> ，写 <code>publishedVersionId</code> 与 status
--	---	--	---

■ 执行读哪一份

默认读 `publishedVersionId` 指向的**不可变快照**——保证一次执行从头到尾用的是同一份图，不会因为有人在编辑而中途变形。

唯一例外：`RunWorkflowDto.launchSource` 为 `'web-studio'` 时读当前草稿。这是给编辑器里的「Run」按钮用的，否则改一行就得发布一次，开发体验不可接受。

■ 三处 fail-closed 校验

- **未发布不能执行**。没有 `publishedVersionId` 直接拒，不会「凑合用草稿」。
- **Agent 节点必须有绑定**。找不到 Agent 定义时直接失败，而不是跳过节点。
- **no_sandbox + stdio MCP 在发布期拒绝**，不留到运行期才炸。

PRINCIPLE

能在发布期发现的问题，绝不留到执行期。执行期的失败要通知人、要清理状态、要写审计，成本高一个数量级。

B4 · EXECUTION ENTRY

runWorkflow() 的九步

src/modules/execution/execution.service.ts

1 · 维护模式闸门 平台维护中直接拒绝	GUARD
2 · 租户内查工作流 必须 published 且有 publishedVersionId	RLS
3 · 输入参数校验 按 input_schema (form / conversation / hybrid 三种采集模式) 归一化	ZOD
4 · 读发布快照 或在 web-studio 来源下读草稿快照	SNAPSHOT
5 · 资源治理准入 租户/工作流暂停? 并发数超? 日执行量超? → 阻断	ADMISSION
6 · 插入 workflow_executions 状态 pending, 同时冻结 definitionSnapshot	WRITE
7 · afterCommit 才入队 jobId = executionId, 队列按 jobId 去重	CRITICAL
8 · Worker initializeSteps 短事务内按快照为每个节点建 execution_steps (pending)	WORKER
9 · NodeScheduler.startExecution 进入 DAG 调度	SCHEDULE

■ 第 5 步为什么在插入之前

被治理阻断的请求**不产生任何执行记录、也不入队**。如果先插入再判断，数据库里会积累一堆「刚创建就失败」的噪声行，监控面板的成功率会失真，用户也会以为「跑过了但挂了」。

阻断路径统一抛

ResourceGovernanceDecisionBlockedException：分钟级 API 限流返回 429 并带 Retry-After / X-RateLimit-*；配额与暂停类返回 409，body 是 RFC 7807 problem+json，并且同步写审计。

■ 第 7 步为什么必须 afterCommit

这是分布式系统里最经典的一类 bug：**事务还没提交，消息已经发出去了**。Worker 抢先执行，按 executionId 去查行——查不到（RLS + 未提交），任务直接失败。

解决方式不是 sleep，也不是重试，而是把入队注册成事务的 afterCommit 回调。配合 jobId = executionId，BullMQ 天然去重，即使回调被触发两次也只会有一个任务。

NOT SOLVED

这仍不是严格的事务性发件箱：提交成功但进程在入队前崩溃，任务会丢。彻底解决要引入 outbox 表 + 轮询补偿，当前没做。

B5 · SCHEDULER

逐层推进，不是遍历

`src/modules/execution/node-scheduler.service.ts`

`DagResolver` 用 Kahn 算法做拓扑排序，检测到环直接抛 `CyclicGraphException`。第 0 层（无前驱的节点）**并行**调度，之后完全由完成事件驱动：

- 01 节点开始前先 `resolveNodeInput()`，从上游 step 的 `result` 按端口取值，写入本 step 的 `input`（**持久化**，便于事后排查「它到底收到了什么」）。
- 02 按 `nodeType` 分发：`agent / chat-agent` → `agent-task` 队列；`plugin` → `plugin-execution` 队列；`text / condition / merge / http / code` 等同步执行。
- 03 完成时 `onNodeCompleted()` **重新读库**，逐个检查后继节点的所有前驱状态：全完成 → 调度；有 `pending` → 等待；条件分支未命中 → `skip` 并级联跳过下游。
- 04 失败时取消所有 `pending / queued / waiting` 步骤，执行整体置 `failed`。

WHY RE-READ DB

调度器不在内存里维护「还差几个前驱」的计数器，而是每次完成都回数据库查前驱状态。

代价是多几次查询；收益是**进程重启后调度状态不丢**，而且多个 worker 并发处理同一执行的不同分支时不会因为内存计数器不同步而卡死或重复调度。数据库是唯一的真相源。

SKIP VS FAIL

条件分支没命中是 `skipped` 不是 `failed`，并且要**级联**——否则下游节点会永远等一个永远不会完成的前驱。

■ 节点分派表（节选）

类型	执行方式	说明
agent / chat-agent	agent-task 队列	经 <code>WorkflowAgentAdapter</code> ；无 Agent 绑定则 fail closed
plugin	plugin-execution 队列	Extism WASM，回调 <code>onNodeCompleted</code>
condition	同步	只调度命中的分支，其余 skip + 级联
loop / iteration	复合上下文	见 B6
http / code / text / merge	同步	纯函数式转换，逻辑抽在同级 util 便于单测
smart-routing	同步	6 种服务端策略纯函数 + 学习子模块（MLP / Elo / KNN）

B6 · COMPOUND NODES

DAG 里怎么放循环

DAG 的定义就是「无环」，而循环天然有环。解决办法是：**循环体不是环，而是一个被反复重置的子图。**

■ 机制

- 调度前先 `filterTopLevel()`，只把顶层节点纳入拓扑排序；带 `parentId` 的节点属于某个复合节点的内部子图。
- `loop / iteration` 节点建立内存中的 `compoundContexts`，把内部子图单独做一次 DAG 排序。
- 每一轮开始前 **reset 子步骤**：清空 `input / result / checkpointData`，然后按序调度。
- 父 step 的 `result` 汇总 `exec-out`、`rounds` 和 `compound` 元信息。

■ 两种语义

类型	终止条件
iteration	遍历完 <code>normalizeLoopItemsInput()</code> 归一后的数组 (foreach)
loop	由 <code>state / next-state</code> 端口驱动的条件循环， 硬上限 <code>maxIterations</code> 默认 100

GUARD
默认 100 轮上限是防「模型判断条件永远为真」导致的无限循环烧钱。这类硬闸门在 AI 系统里必须有，且不能只靠提示词约束。

BREAK

优先级最高：先执行本轮的 `result` 节点保住已产出的数据，再 `finalize` 整个复合节点。不会丢掉最后一轮的成果。

CONTINUE

丢弃本轮输出，直接推进下一轮。语义与编程语言一致，降低用户心智负担。

OUTPUT MODE

`none`（不收集）/ `last`（只留最后一轮）/ `collect-array`（聚合成数组）。默认收集会撑爆 `result` 体积，所以要显式选。

INTERVIEW

「为什么不直接在图里画一条回边？」——回边会让拓扑排序失效，也让「某个节点跑到第几轮」这件事无法在 `execution_steps` 里表达（一个节点只有一行）。用父子层级 + 每轮 `reset`，可以让每个内部节点保持单行记录，同时把轮次信息存进父节点的 `rounds`。代价是丢失了历史轮次的中间产物，只能靠事件流回看。

B7 · CHECKPOINT & RESUME

崩溃之后从哪继续

`CheckpointService.saveCheckpoint()` 在每个节点完成后**合并式更新** `execution_steps.checkpointData`，而不是整体覆盖：

字段	内容
output	节点结果
completedAt	ISO 时间戳
dagState	{ completedNodes[], pendingNodes[] }
session	Agent 运行时会话快照（ workflow 侧持久化位置）
partialContent	流式输出的已收到部分
toolCalls	工具调用及其权限状态
round / subAgentStreams	复合轮次与子 Agent 流

`attemptCount` 单独计数，不混在 `checkpoint` 里，避免合并语义把重试次数覆盖掉。

resume 的两种模式

整体续跑

把 `failed / cancelled` 的步骤重置为 `pending`，`attemptCount=0`，执行置 `running`，投 `resume-execution`。调度器跳过 `completed` 步骤。

从指定节点重跑

带 `fromNodeId` 时按 BFS 找出该节点**及全部下游**，一起重置并清 `result`。上游成果保留。

CONSTRAINT

`POST /executions/:id/resume` 只接受 `failed` 状态。`running` 中的执行不允许续跑，否则会出现两个调度器同时推进同一张图。

HONEST LIMITATION

检查点保证的是「**调度层面**可恢复」，不是「副作用可恢复」。如果一个节点已经发出了 HTTP POST 然后进程挂了，重跑会再发一次。要做到真正的 `exactly-once`，需要节点级幂等键，目前只有部分节点（如插件执行按 `step` 记录）具备。这是可以主动承认的设计边界。

B8 · REALTIME

带单调游标的事件流

■ 事件信封

```
ExecutionEvent<T> {
  eventId: number // 每执行单调递增
  event: string // execution.node.*
  timestamp: string
  executionId: string
  tenantId: string
  data: T
}
```

房间命名 `execution:{tenant}:{execution}`，Redis adapter 负责跨进程。每个执行维护一个 **500 条环形缓冲**。

■ 断线重连的三种情况

- 01 缓冲够。**客户端带 `lastEventId` 订阅，服务端从缓冲区取增量补发。
- 02 缓冲不够。**`getEventsSince()` 返回 `null`，服务端改发一份完整 **state snapshot** + 活跃 **step** 的初始事件，客户端整体覆盖本地状态。
- 03 没落后。**当前 `eventId` $\leq$ 客户端游标，什么都不发。

终态处理很讲究：先 `flushExecutionQueue()` **排空积压事件**，再立即**广播终态**，**30 秒后才清理 counter 和 buffer**。这样刚好错过终态的客户端还有 30 秒窗口能拉到完整结局。

■ 背压：令牌桶 + 每执行队列

01 令牌充足 直接 emit	02 令牌耗尽 进入该执行的专属队列	03 队列满 500 丢最旧的事件，保新不保全	04 100ms drain 定时排空；终态前先整体 flush，保证结局不被积压挡住
------------------------------	---------------------------------	--------------------------------------	--

WHY DROP OLDEST

LLM 流式输出的 token 事件是「越新越有用」。丢最旧的 chunk 只会让用户看到内容跳一下，丢最新的会让界面卡在过去。而终态事件的做法不是「插队」——是**先整体 flush 排空队列，再广播终态**，保证结局不被积压挡住。

DRIFT

`/agent-conversation` 的前端 store 虽然有 `lastEventId` 字段，但 `subscribe` 时只发了 `conversationId / tenantId`，事件处理也没更新游标——**对话侧的回放契约实际未接通**。这是必须主动交代的实现缺口。

B9 · TRIGGERS & NOTIFICATION

三种入口，一个出口

CRON

BullMQ repeat pattern + timezone, 持久化 `nextFireAt`, 启动时同步所有 enabled 定时器。执行走 `triggerType='system'`。

WEBHOOK

@Public 的 `/webhooks/:token`。signed 模式做 `HMAC-SHA256(timestamp + '.' + rawBody)` + `timingSafeEqual`, 时间戳容差 300 秒, 可配 IP 白名单。

API EVENT

POST `/api-events` 受完整鉴权保护。
`EventSourceAdapterRegistry` 按 source 分发到 `GithubWebhookAdapter` (HMAC 验签) 或 `GenericEventAdapter`。

■ 失败也要留痕

验签失败不是静默 401, 而是往 `workflow_trigger_history` 写一条 `signature_failed`。不匹配任何 trigger 的事件写 `skipped`。

理由: webhook 是最容易「配错了但没人知道」的集成点。有历史记录, 用户能自己排查; 没有的话, 所有问题都会变成支持工单。

LIMIT

每个工作流最多 10 个 trigger, 且创建 trigger 要求工作流已发布。

■ 通知扇出

执行状态变化通过 `EventEmitter2` 发出, `NotificationListener` 监听 `execution.status.changed` (completed / failed) 和 `intervention-required`, 在**独立的租户事务**里解析收件人 (workflow 编辑者等), 写 `notifications` 表后投递队列。

Processor 再按用户的 `notification_preferences` 决定走 `in_app` (Socket.IO /notification namespace 推 `notification.new` + 未读数)、`email` 还是 `push` (FCM)。

WHY SEPARATE TRANSACTION

通知失败不能回滚执行。用独立事务把两件事解耦, 通知是「尽力而为」, 执行是「必须正确」。

PART C

C

智能体

Agent 有自己的定义、版本、对话、运行时、记忆和自我修改能力。这一部分讲清楚：谁负责持久化、谁负责活的运行态、模型想改自己的配置时怎么拦。

C1-C2

边界 · 版本漂移

C3-C4

双运行态 · Skill

C5-C6

记忆图谱 · 自进化

C1 · AGENT VS WORKFLOW

一份逻辑，两种入口

Standalone 对话

REST + /AGENT-CONVERSATION

用户直接和 Agent 聊，长期存在、可继续、可重启到新版本

Workflow 节点

AGENT-TASK 队列

DAG 里的一步，有明确输入输出端口和终点

Sub-Agent

递归复用同一桥接层

父 Agent 把子任务派发给子 Agent，共享同一 runtime 抽象

■ 三条入口收敛到同一个接口

```
interface IAgentRuntime {
  createSession(config: AgentRuntimeConfig)
  loadSession(snapshot)
  prompt(input) / cancel()
}
```

AgentRuntimeConfig 含 modelConfig / tools / knowledgeBindings / subAgents / memoryInstanceIds / skillIds / outputSchema / nativeToolPolicy

两个实现：**SandboxAgentAdapter**（Firecracker guest）和 **InProcessAgentAdapter**（进程内）。上层三种入口都只依赖这个接口，所以换运行时不影响业务代码。

■ Sub-Agent 的端口语义

端口	语义
system-prompt-in model-in / schema-in	override : 覆盖父配置
tools-in / skills-in sub-agents-in / knowledge-in / memory-in	extension : 在父配置基础上追加
sandbox-in	不提供

■ 为什么 sub-agent 不能自带沙箱

如果每个子 Agent 都能开自己的 microVM，一次对话可能瞬间拉起十几个虚拟机，成本和调度都失控；而且父子之间的文件系统割裂，「父让子改一下这个文件」这种最常见的协作就做不了。

所以规则是：**子 Agent 并入父的沙箱**，并且如果父是 sandbox 模式、子是 no_sandbox，子会被降级为只读的 native tool policy；反过来 no_sandbox 父调用 sandbox 子**直接拒绝**——进程内运行时没有能力托管一个 microVM 的生命周期。

PRINCIPLE

能力只能收窄，不能在调用链中途放宽。这是权限设计里最省事也最不容易出错的规则。

C2 · VERSION DRIFT

对话跨越了发布

一个真实存在的麻烦：用户昨天和 Agent 聊了 20 轮，今天你发布了新版本改了系统提示词。这个对话继续时，应该用旧配置还是新配置？

设计选择

对话表只关联 agentDefinitionId，不永久绑定版本。当前 runtime 实际加载的版本记录在 agent_conversations.metadata.execution.loadedPublishedVersionId。

继续执行时 worker 比对它和定义当前的 publishedVersionId：不同就**自动刷新**——清理旧 Memory session、释放 direct sandbox、清空 sessionId，以 idle 状态用新配置重建。

手动路径

POST /agent-conversations/:id/restart-latest-version 显式刷新。它**只在当前对话内**切换配置，不新建会话，历史消息和会话级 remembered policy 全部保留。

早期实现是「新建一个对话」，用户会丢失上下文并抱怨对话列表被刷屏，后来改成原地重启。

Agent 侧的版本模型（与工作流对称）

对象	要点
agent_definitions	canvas nodes/edges/viewport、runtimeMode、草稿 version (OCC)、status、publishedVersionId
agent_versions	(agentDefinitionId, versionNumber) 唯一；快照含 canvas、runtimeMode、sandboxConfig、inputSchema、memoryInstanceIds、releaseNotes
写入保护	画布保存在 withAgentWriteLock 内递增草稿 version
发布	先清所有旧版本的 publishedAt，再发布指定历史版本或从草稿新建快照，最后原子更新定义
归档	同时给所有版本打 archivedAt

WHY NOT PIN

「把对话钉死在创建时的版本」也是一种设计，但它会让 bug 修复永远无法到达老对话。选自动刷新 + 显式重启，是把「配置正确性」排在「绝对可复现」之前——因为对话本来就不可复现（模型有随机性），而工作流执行必须可复现，所以工作流那边选的是钉死快照。同一个问题，两种实体给了相反的答案，理由是**可复现性要求不同**。

C3 · DUAL RUNTIME

沙箱与进程内

创建 Agent 时显式固定 `runtimeMode = sandbox | no_sandbox`，之后不可改。

维度	SANDBOX	NO_SANDBOX
执行位置	Firecracker microVM 内的 pi-coding-agent	服务进程内 pi-agent-core
调用链	SandboxAgentAdapter → manager mTLS proxy → guest	InProcessAgentAdapter → PiAgentCoreAdapter
Coding 工具	完整：读写文件、终端、执行命令	不提供
Workspace	真实文件系统 + 快照	无
MCP	HTTP + stdio	仅 HTTP，stdio 在发布期 fail-closed
启动开销	microVM 冷启动 + 网络配置	近乎为零
Skill / 知识库 / Memory / 自进化	两者都支持	

KEY RATIONALE

no_sandbox 禁 stdio 不是「懒得做」。stdio MCP 意味着在**服务进程里 spawn 任意本地命令并注入环境变量**——这等于把 Node 进程的全部权限交给一段配置。HTTP MCP 仍然是远程调用，受网络与凭证治理约束。

持久化层与活运行态的分工

PiAgentCoreAdapter · 易失

只管内存里活着的 Agent 实例：订阅事件、prompt / abort、工具调用分发、权限门。进程重启即消失。

InProcessAgentAdapter · 持久

维护 session 映射与快照。工作流写 `execution_steps.checkpoint_data.session`；对话写 `acp_conversation_sessions.session_snapshot`，并把 `user_message / plan / message_chunk / tool_call / decision` 追加进 `replay_entries`。

这样切的原因很直接：**pi-agent-core 是一个通用运行时库，它不该知道你的数据库长什么样**；而业务侧必须能在进程崩溃后重建对话，不能依赖内存。代价是要维护两套状态并保证它们一致——这是本项目里最容易出 bug 的接缝之一。

ZOD ↔ TYPEBOX

`tool-schema-converter.ts` 做双向转换：AI SDK 侧用 Zod / JSON Schema，pi-agent-core 侧用 TypeBox。支持 object / array / enum / optional / nullable。不做转换的话，工具参数就无法在 pi 侧被校验。

HONEST

`stream-fn.adapter.ts` 里的 `createVercelStreamFn`（把 AI SDK `streamText().fullStream` 桥成 pi 的 `start/text/thinking/toolcall/done` 事件）目前在**生产代码里没有调用点**，属于已写好但未启用的兼容层。别把它讲成「已经在跑」。

C4 · SKILL SYSTEM

把「怎么做」外置成文件

Skill 是一份 SKILL.md（YAML frontmatter + Markdown 正文）加可选附件。它解决的问题是：同一套方法论（比如「怎么做 code review」）不该被复制粘贴进每个 Agent 的系统提示词里。

■ 存储与注入

STORE MinIO + Postgres 对象键 <code>tenants/{tenantId}/skills/{skillId}/</code> ; <code>skills</code> 表存租户、slug、frontmatter、版本、 <code>is_builtin</code> 、 <code>file_count</code>	RESOLVE SkillResolverService 按租户过滤 enabled skill，生成转义后的 <code><available_skills></code> XML	INJECT A 提示词通道 小内容连正文一起注入；大内容只给摘要，避免上下文爆炸	INJECT B 文件通道 sandbox 由 <code>pi-config-input.builder.ts</code> 下发 <code>skills[].files</code> 真实多文件
--	---	---	--

■ 六个内置 Skill

`code-review` · `documentation` · `test-generation` · `refactoring` · `debugging` · `self-evolution`

由 `pnpm db:seed` 的 `seedSkills()` 按 slug 幂等 upsert，用哨兵 UUID `00000000-...-0000` 标记系统内置，`isBuiltin=true` 不可删除。

NOTE

`self-evolution` 本身就是一个 Skill——教 Agent 怎么用那五个自进化工具。能力和使用说明用同一套机制分发。

■ 为什么要两条通道

`no_sandbox` 没有文件系统，只能靠提示词；`sandbox` 有真实工作区，把附件压成一个字符串塞进 prompt 既浪费 token 又让 Agent 无法用 `grep` 之类的工具去检索。

所以同一份 Skill 数据，按运行态选择投递形态：**提示词是「知道有这件事」，文件是「能真的读它」**。这也顺带解释了为什么大 Skill 只注入摘要——模型看到摘要后，如果需要细节，会自己去读文件。

COST

代价是同一个 Skill 在两种运行态下模型看到的东西不完全一样，行为可能有细微差异。这是刻意接受的不一致。

C5 · AGENT MEMORY

不是向量库，是图

Memory 用 PostgreSQL 存图谱，检索用 PostgreSQL 全文检索（`to_tsvector` / `to_tsquery` / `ts_rank`），**没有走 Qdrant**。这是个会被追问的选择。

■ 七张表

表	作用
<code>agent_memory_instances</code>	实例与 OCC 版本
<code>memory_nodes</code>	节点内容与披露级别
<code>memory_edges</code>	父子关系，唯一结构关系，EdgeService 防环
<code>memory_versions</code>	内容版本、deprecated、reviewStatus
<code>memory_paths</code>	路径索引
<code>memory_glossary_keywords</code>	术语表
<code>memory_sessions</code>	按 <code>execution_id</code> 或 <code>agent_conversation_id</code> 绑定

■ 为什么是图 + FTS

- **记忆需要结构**。「项目 A → 模块 B → 约定 C」这种层级关系，向量检索表达不了，但对 Agent 复用知识至关重要。
- **记忆需要版本与审阅**。`memory_versions` 带 `reviewStatus`，错误的记忆可以被标记废弃而不是物理删除——这是向量库很难做的。
- **记忆需要披露控制**。按 disclosure 级别过滤，不同场景看到不同深度。
- **规模不同**。知识库是几万个文档 chunk，记忆是几百条精炼结论。FTS 足够，省一个外部依赖。

■ 融合读取

`MemoryFusionService` 按 session priority 融合「读 / 搜 / 写 / boot protocol」四类操作。boot protocol 是会话启动时先注入一批高优先级记忆，让 Agent 一开始就有上下文，而不是等它自己想起来去搜。

■ `/memory` namespace

WsJwtGuard 鉴权，房间 `memory:{tenantId}:{instanceId}`，`lastEventId` 最多回放 1000 条，背压每实例 500 / 100ms drain。事件：
`memory.node.*`、
`memory.version.created|rollback`、
`memory.review.submitted`。

C6 · SELF-EVOLUTION

让 Agent 改自己，但要人点头

五个低层工具

工具	性质
query_state	读：当前自身配置
query_resource_pool	读：可用的模型 / 工具 / Skill / MCP / 工作区
propose_change	提案：产出 diff，不落盘
apply_change	写：需要审批
create_resource	写：需要审批

提案对象包含 target、baseVersion、节点/边操作、diff、分类、risk、requiresConfirmation。分类覆盖「自编辑 / 改别的 Agent / 改 Workflow / 改 Skill / 改 MCP / 改 Model / 改 Workspace / 改 Sandbox」。

awaiting_permission 是 tool-level 状态，step 本身保持 running。

因为等待的是一次工具调用，模型的这一回合还在进行中。如果把整个 step 置成 waiting，恢复时就得重建整个 agent loop 的状态机；只冻结那个工具调用，用户点了「同意」之后 loop 可以原地继续。

SelfEvolutionPermissionService 把高风险请求登记为 pending，用 Redis 跨进程共享——因为发起 prompt 的 worker 和接收审批 HTTP 请求的 server 通常不是同一个进程。

记住决定

用户可以按 rememberScope = conversation_category 记住 approve / deny，策略写进 agent_conversations.metadata.rememberedPolicies，刷新版本时保留。

粒度选「会话 × 分类」而不是「全局」或「单次」：全局太危险（一次同意等于永久授权），单次太烦（同一类操作要点二十次）。

审批接口

POST /agent-conversations/:id/tool-permissions/:toolCallId/resolve

工作流侧对称：POST /executions/:executionId/steps/:stepId/tool-calls/:toolCallId/resolve

SCOPE

目前只有自进化的两个写工具会触发 awaiting_permission。普通工具（读文件、搜索、HTTP）自动放行，否则人会被打断到无法使用。

RELATED

配套还有一条离线优化闭环：optimization_suggestions 表存 model_downgrade / timeout_adjustment / tool_pruning / autonomy_upgrade 四类建议，BullMQ upsertJobScheduler 注册 UTC 0 2 * * 1 周任务，读近 28 天 agent_execution_records，遥测不足 20 条的节点跳过。apply 时双重 guard：工作流 version OCC 防覆盖并发编辑 + WHERE status='pending' 防重复消费，任一失败返回 409。注意 OptimizationSuggestionsPanel 目前没有生产挂载点，只被测试引用。

PART D

D

隔离与生态

这一部分回答同一个问题的两半：**怎么安全地跑别人写的代码**。Agent 生成的代码进 Firecracker 微虚拟机，第三方插件进 Extism WASM，生成的应用要过八道门禁。

D1-D3

microVM · 网络身份 · 租约

D4-D5

ACP 协议 · 回调中继

D6-D9

插件 · 市场 · 生成应用

D1 · FIRECRACKER

为什么不是容器

Agent 会执行模型生成的任意代码。容器共享宿主内核，一个内核漏洞就是一次逃逸。Firecracker 给每个沙箱一个**独立内核**，逃逸需要先打穿 KVM/virtio——**攻击面从「整个系统调用接口」收窄到「虚拟设备接口 + 虚拟机监控器」**，而启动开销远低于完整虚拟机。（本项目没有做过冷启动耗时基准测试，不要在面试里报具体毫秒数。）

server / worker 普通权限，只持有 manager client 证书	mTLS CLIENT
runtime manager (Go, 单例 privileged) 独占 /dev/kvm、/dev/net/tun、host cgroup	CONTROL PLANE
jailer chroot + 降权 + namespace，启动 Firecracker 进程	SANDBOX LAUNCHER
CNI + netns + veth + tc + nftables 每 VM 独立网络命名空间	NETWORK
只读 rootfs + 可写 mutable ext4 disk cgroup v2 cpu.max / memory.max / pids.max，SMT=false, MMDSv2, seccomp	VM
guest pi-coding-agent + Fastify + agentloom-guestd，视为完全不可信	UNTRUSTED

■ 创建流程

- internal/manager/manager.go 先校验规格 (CPU 0.5-4、内存 256-4096MB、磁盘 1-10GB)，然后：
- 01 Ext4DiskManager.Ensure：mke2fs 建 mutable.ext4，已存在则 e2fsck
- 02 CNI Provision: 建 netns / veth / TAP
- 03 launcher + jailer 启动 Firecracker
- 04 GuestChecker 健康探测 (HTTPS + IP SAN 校验)

■ guest 暴露的接口

来源	接口
Fastify (TS)	POST /v1/session、/v1/prompt (SSE)、/v1/abort、PTY buffer / 写入 / resize / kill
guestd (Go)	/v1/runtime/archive (tar)、/files GET/HEAD/PUT、/exec + output/wait/kill、stats、processes
全部只能经 api/proxy.go 的 /v1/vms/{id}/guest/{path} 访问，manager 注入自己持有的 bearer token 并按分配 IP 建连。	

GUEST-SIDE GUARDS

路径只允许 /workspace 或 /tmp，用 EvalSymlinks 防逃逸；文本 ≤ 10MiB 且拒绝含 NUL 的二进制；tar 上限 10GiB 并防 traversal / symlink；exec 限制命令、参数与环境变量，显式禁 LD_PRELOAD / LD_LIBRARY_PATH / NODE_OPTIONS。

D2 · NETWORK IDENTITY

假设 guest 会撒谎

网络隔离最常见的失败模式不是「忘了加规则」，而是「规则按 IP 写，而 guest 可以伪造 IP」。所以这里的每一层都在**验证身份**，不只是过滤地址。

L1 · GUEST 内核禁用 IPv6 堵掉「v4 规则齐全、v6 完全放行」这个经典旁路	L2 · VETH tc ingress 校验 host 侧 veth 上同时校验源 IPv4 + MAC，以及 ARP 的 sender IP + MAC；其余帧 <code>matchall drop</code>	L3 · NFT 拓扑拒绝 拒绝 sibling VM 互访、private/reserved CIDR、非 relay host；只放行 relay 端口 18080 入站	L4 · TLS IP SAN 校验 manager 连 guest 时按分配 IP 校验证书 IP SAN，禁止 <code>ServerName</code>，不用共享 DNS SNI
---	--	--	--

■ 这套设计防的攻击

- 伪造源 IP / MAC 冒充其他 VM
- ARP 欺骗劫持同网段流量
- IPv6 绕过 v4 规则
- 横向移动到 sibling VM
- 访问宿主、云 metadata (169.254.169.254)、公司内网
- 伪造回调制造 SSRF

■ 两个独立信任域

manager ↔ server/worker 是一套 PKI；manager ↔ guest 是**另一套 guest CA**。server/worker **不持有 guest CA 私钥**，guest 也不持有应用侧 client 证书。

这意味着即使应用服务被完全攻陷，攻击者也签不出能让 manager 信任的 guest 证书；反过来 guest 被攻陷也无法直接伪装成 server 去调用 manager 的控制接口。

EGRESS

出网默认只允许公网（private CIDR 默认集合全拒），需要访问内网必须显式配置 allowed private CIDR 白名单。guest 出网做 IP masquerade。

WHY BOTHER

如果只用 nftables 按 IP 过滤：guest 里的 root 可以直接改自己的 IP/MAC，冒充另一个租户的 VM 或宿主，规则就形同虚设。**tc 层的 IP+MAC+ARP 三重绑定，是把「网络地址」变成「不可伪造的身份」的关键**。这是这套沙箱最值得讲的一个细节。

D3 · LIFECYCLE & LEASE

谁在用这个盘

■ sandbox_sessions

双外键： `execution_id` OR `agent_conversation_id`，加 CHECK 约束「至少一个非空」。这让同一套沙箱基础设施同时服务 workflow 和 Agent 对话，而不用建两张几乎一样的表。

字段还包括 `sandbox_node_id`、`tenant_id`、不透明的 `runtime_handle`、`status`、`config` JSON、`workspacePath`。

模式	销毁语义
session	最后一个直接消费者完成后销毁
persistent	解绑回到 ready/idle；stop / timeout / 过期只停 VM， 保留 handle 与 ext4 盘 ；只有显式 <code>deleteSandbox()</code> 才删盘

工作流侧粒度是 (`execution_id`, `sandboxNodeId`)，同一次执行内多个节点引用同一 persistent 沙箱时，用 `config.activeBindings` 计数追踪。

■ 冷恢复：fail closed

manager 重启后 `Recover()` 扫描残留：persistent 且 `e2fsck` 通过 → 标记 `stopped` 可重启。

但只要清理 `jailer` / `cgroup` / `PID` / `netns` 中**任意一项失败**，就**保留内部 handle 并置 failed，拒绝继续**。宁可留下一个需要人工处理的资源，也不冒「以为清干净了其实还在跑」的风险——那会导致两个 VM 抢同一块可写磁盘。

■ Workspace 租约：防双挂载

`workspace_runtime_leases`： `workspace_id` 唯一 + `sandbox_session_id` 外键 + `fencing_token` + 过期时间。

恢复工作区前先 acquire 租约，恢复后 `assertHeld`；stop / destroy / timeout / detach 时回写快照，最后一个 mount 离开才释放。

FENCING

没有 fencing token 的分布式锁是假锁：一个卡住的旧持有者恢复后仍会写入，覆盖新持有者的数据。带单调递增 token 后，存储侧可以拒绝低版本写入。

■ Workspace 快照

`workspace_snapshots`： `storageKey` 指向 MinIO 里的 `snapshot.tar`，`status` 为 `creating|ready|archived|deleted`。

读取接口：GET `/workspaces/:id/tree`、`/preview/*`、`/raw/*`，PUT `/:id/files/*`。Studio 侧可预览文本 / 图片 / PDF，其他类型给下载兜底。

D4 · ACP GATEWAY

把平台接进编辑器

ACP (Agent Client Protocol) 是基于 JSON-RPC 2.0 over stdio 的协议，让外部客户端（编辑器、IDE）把 AgentLoom 当作后端 Agent 使用。入口是独立的 `src/acp-stdio.ts`（`pnpm start:acp:stdio`），逐行异步读取，用 `writeChain` 串行化 stdout 写入避免交错。

方法集

域	方法
握手	<code>initialize</code> （仅接受协议 2026-02-18）、 <code>authenticate</code>
会话	<code>session/new</code> 、 <code>session/prompt</code> 、 <code>session/cancel</code> 、 <code>session/load</code> 、 <code>session/update</code>
文件	<code>fs/read_text_file</code> 、 <code>fs/write_text_file</code>
终端	<code>terminal/create output wait_for_exit kill release</code>
反向请求	<code>server → client</code> 的 <code>session/request_permission</code>

能力协商

canonical 名是 `readTextFile` / `writeTextFile`（兼容 legacy `read` / `write` 别名）。

terminal 能力只有在 client 同时声明 `terminal.create` 和 `terminal.output` 时才暴露——半支持的客户端拿不到一半能力，避免运行到一半才发现对端不会读输出。

WRITE GATE

写文件前必须先发起官方 `session/request_permission` 并等到响应。`session/cancel` 会清理所有 pending permission，防止取消后请求悬挂。

文件边界

`AcpFilesystemSandboxService` 只解析 `/workspace` 下的路径，相对路径基于 `session cwd`；`traversal` / 越界一律拒绝。`runtime` 绑定按 `tenant + execution/conversation` 查 active 的 `ready/busy` 会话，最终 `realpath` 由 guest 侧再校验一次（不信任 server 侧的解析结果）。读写上限 10MiB，含 NUL 的二进制拒绝。

终端边界

- 只允许 `server_sandbox` 模式
- 命令 `basename` 黑名单：`bash sh zsh fish sudo su docker kubect1`
- 拒绝 shell 注入、`rm -rf` 危险目标、`cwd` 与赋值型 flag 的路径越界
- 每 session 并发 5，ring buffer 默认 1MiB，offset 过旧返回 `trimmed`
- lifetime 300s 自动 TERM；退出后再取 output 返回稳定错误
- manual kill 与 cleanup kill 都写正式审计

COLD RECOVERY

`session/load` 在冷恢复场景 **fail-closed**：如果无法确认原会话的沙箱与权限上下文，宁可报错让客户端重开，也不「尽力恢复」到一个权限状态不明的会话。

D5 · CALLBACK RELAY

guest 要回头找人，但不能知道人在哪

Agent 在沙箱里调用工具时，有些工具必须回到创建这个会话的**那一个** server/worker 进程执行（权限确认、session-local 远程工具）。直接给 guest 一个内部 URL，等于把内网地址和令牌交给不可信代码。

01 server 发起 prompt 请求里带真实 permissionCallba ckUrl 和上游 token	02 manager 改写 proxy.go 的 rewriteCallback s 换成不透明 relay URL + 32 字节随机 token	03 guest 只见 relay 看不到任何内部主机名或上游凭证	04 manager 校验 来源 RemoteAddr 必须等于该 guest 的分配 IP；再校验不透明 header	05 代持转发 manager 换回真实 upstream token 转发给白名单主机
---	--	---	--	--

callbackRegistry 的约束

项	值 / 理由
allowedHosts	只允许 server / worker，别的地址一概拒绝（反 SSRF）
token	32 字节随机，不透明，与业务无关联
TTL	24 小时
上限	10,000 条，防注册表被打爆
来源校验	RemoteAddr = guest IP，双重绑定

guest 侧发出的 permission 回调请求 30 秒无响应就 abort 并**按拒绝处理**（`PERMISSION_TIMEOUT_MS`，`deploy/sandbox/src/event-stream.ts`）——超时不能默认放行。注意服务端等待人工审批用的是另一个 300 秒超时，见 F4 Q21。

三个理由叠加：

- ① guest 在隔离网段，本来就不该有到应用内网的路由；
- ② 直接给 URL/token 等于把内部拓扑和凭证泄露给不可信代码；
- ③ 必须精确回到**创建 session 的那个进程**——它内存里有活的 runtime，随便负载均衡到另一个副本会找不到会话。

D6 · PLUGIN PACKAGE

规范化归档签名

`.alp` 本质是一个 ZIP。问题在于：**ZIP 的字节表示不唯一**——条目顺序、压缩级别、时间戳、额外字段都会变。对整个文件签名，任何一次无害的重打包都会让签名失效。

■ canonical descriptor

`plugin-sdk/src/signing/archive.ts` 读取所有 entry，规范化路径并拒绝 `.` / `..` / 重复路径，然后构造：

```
{
  manifest: deepSort(manifest 去掉
    signature / contentHash /
    developerKeyFingerprint),
  files: [ [path, sha256(content)]
  ... ]
  // 按 path 排序
}
```

对这个 JSON 做 SHA-256，再用 **RSA-PSS / SHA-256 / saltLength=DIGEST** 签名。验签侧用 `AUTO saltLength`。

只绑定业务内容，不绑定容器格式。重新打包不影响签名有效性。

CLI REALITY CHECK

`plugin-cli` 的 `keys` 命令**目前只实现了** `generate` (RSA 2048/3072/4096，私钥落盘 0600)，没有 `list`；`publish` 也只做签名 + 自验 + 提示走 Web 上传，**没有真正的 HTTP 上传**。文档里写的完整命令集与实现有出入。

■ 服务端注册链路

- 01 multipart 上传 `.alp` ($\leq 50\text{MB}$)，必须带三元组：`signature`、`contentHash`、`developerKeyFingerprint`
- 02 按 `org + fingerprint` 找 **active** 的开发者公钥
- 03 重算 canonical descriptor 验签 + 比对 `content_hash`
- 04 通过后才上传 MinIO 并写 `plugins` 表

`plugin-signature.service.ts` 校验公钥本身：**拒绝私钥**（防手滑上传）、必须 RSA、位长 ≥ 2048 ，指纹 = SPKI DER 的 SHA-256。

KEY LIFECYCLE

`plugin_developer_keys` 唯一键 `org + fingerprint`，状态 `active | revoked`，撤销带 `revoked_at`。API 在 `/plugins/developer-keys`，creator 及以上可用。

D7 · WASM & ECONOMICS

三十秒，256MB，没网

■ 平台硬限制（不可被插件放宽）

项	值
timeoutMs	30,000
maxMemoryPages	4,096 (≈ 256MB)
allowedPaths	{ } 空——无文件系统
useWasi	false
runInWorker	true (Extism 只有在 worker 里才支持 timeout 与 allowedHosts)
网络	默认无；只有 manifest 声明 network:outbound 且提供 sandbox.allowedHosts 数组时才开放

运行时配置**只能进一步收紧** timeout / memory / hosts，永远不能放宽。这是「能力只能收窄」原则在插件侧的复用。

■ 执行 worker 的事务边界

`PluginExecutionWorker.process` 全程在 `runInTenantTransaction(tenantId)` 内：step 置 running → 找 active 插件 → 从 MinIO 下载 WASM → 构造 config → 执行 → 归一化输出 → step 置 completed/failed，最后回调 `NodeSchedulerService.onNodeCompleted/onNodeFailed`。

用量记录 `plugin_usage_records` 虽然是 `await` 的，但整段包在只 `warn` 的 `try/catch` 里：**记账失败不会让用户的执行失败**。

■ 收益分成

70%

开发者毛收入

×15%

上架佣金（从毛收入扣）

59.5%

开发者净收入

30%

平台份额

`PluginEarningsService.calculateSettlementShares` :

```
gross = total × 0.70 , commission = gross × 0.15 , developerShare = gross - commission , platformShare = total × 0.30 。
```

payout 状态机： `pending` → `processing` → `completed` | `failed` 。

`IDEMPOTENCY` ×2

`EarningsSettlementWorker` 在租户事务内聚合周期用量，跳过 0 值与已有记录；service 侧再用 `onConflictDoNothing(plugin, period)` 兜底。**钱相关的写入必须有两层幂等**——一层在业务逻辑，一层在数据库唯一约束。

D8 · MARKETPLACE & SHARE

三条分发路径

MARKETPLACE

审核制上架。
`marketplace_listings` 的 `listingType` 为 `workflow / plugin`, `pricingModel` 为 `free / per_execution`, `workflowVersionId` 与 `pluginDbId` 都可空但有 `CHECK` 强制二选一绑定。状态 `pending_review` → `review_failed` | `listed` → `unlisted`。

SHARE LINK

点对点短链。管理端 `/workflow-shares`、`/agent-shares`，公开只读 `/s/:token`。创建要求已发布，公开读取从快照返回 `nodes/edges/viewport`，并原子递增 `view_count / copy_count`。

DISCOVER

`/discover` 复用 Marketplace 已上架内容做公共浏览入口，**不承载直链分享池**——避免出现一个没人审核的公开内容池。

■ Agent 导入为什么要回执

`POST /agent-shares/:token/import` 返回 `summary + report[]`，而不是一个 ID。

因为 Agent 会关联知识库、Memory、MCP、Skill、工作区——这些**跨租户不可能原样搬**。导入时 `workspace` 被清空，模型等外部资源标记为 `needs_rebind`。用户必须知道「哪些搬过来了、哪些需要你自己接」，否则会拿到一个静默半残的 Agent。

Workflow 侧简单些：直接按 `share_token` 克隆导入 (`cloneDefinitionWithNewIds()`)。

■ 来源分类

`resource_source_records` 统一记录 `workflow / agent / knowledge / memory / mcp / skill` 六类资源的来源：`origin` 与 `currentKind` 取值 `manual` | `share_imported`。

`POST /resource-sources/:type/:id/convert-to-manual` 只切换**当前分类，不复制新资源**，并保留 `origin`。这样「这东西是我自己建的还是导入的」永远查得到，同时用户又能把导入的东西「认领」为自己维护。

EXPORT

导出用 `agentloom-workflow-v1` 信封 + `sanitizeDefinition()` 递归剥离敏感信息；导入含 `Zod` 校验。创建工作流支持 `template_slug / share_token / marketplace_listing_id` 三种克隆源，互斥。

D9 · GENERATED APP

八道门禁

自然语言 → 可发布应用。每一道 Gate 的产物都写进 `generated_app_gate_runs` 作为证据。

Gate 0 · AppSpec 把自然语言变成结构化规格	SPEC
Gate 1 · 架构计划 模块、数据流、依赖	PLAN
Gate 2 · 静态契约 接口与类型先定死	CONTRACT
Gate 3 · 受控工作区 生成源码 + 构建 + 单元测试	BUILD
Gate 4 · 集成测试 模块间真实联调	INTEGRATION
Gate 5 · 浏览器验收 真实驱动 UI 跑验收场景	ACCEPTANCE
Gate 6 · 独立验证 换一个 verifier 重新检查，不信任生成者的自评	VERIFY
Gate 7 · 发布候选 全部阻断项通过才允许 public share	RELEASE

为什么要 Gate 6

让生成代码的模型自己判断「我写得对不对」，通过率虚高。Gate 6 用**独立的验证者**重跑检查，相当于把「自测」和「验收」分离。失败可以走 repair / retry 循环，修复尝试记在 `generated_app_repair_attempts`。

公开运行时的清洗

`generated-app.public-sanitizer.util.ts` 剥离 token / secret / API key、内部 gate 与 plan 快照、插件 ID、绝对路径，并限制递归深度、字段数、数组长度和文本长度（防 DoS 式膨胀）。

公开页 CSP `default-src 'none'`，只暴露运行时表单与报告/执行交接，**不暴露画布和证据**。

CODE ORGANIZATION

`GeneratedAppService` 只保留 DB / 事务 / 公开运行时 / 插件绑定 / Gate 编排；各 Gate 的计划构建在 `plan-builders/`，校验、清洗、runtime binding、AppSpec 转换在同级**纯函数 util**。这样最复杂的判定逻辑可以脱离数据库与队列单独测——服务端 80% 覆盖率门槛就是靠这种拆法达成的。

PART E

E

平台面

前端怎么切、类型兼容为什么用 Rust、审计为什么是两张表、密钥为什么服务端解不开、移动端为什么不做编辑器、部署为什么只有一个特权进程。

E1-E4

Studio · 画布 · 类型引擎 · 加密

E5-E7

审计 · 配额 · RAG 与 MCP

E8-E9

移动端 · 部署与备份

E1 · STUDIO

服务端状态与本地状态分开管

■ Feature-Slice 三层

层	允许放什么
app/	应用壳、providers、 手写路由树 、认证守卫
features/	每个切片自带 api / keys / queries / hooks / stores / components / types
shared/	ky 客户端、QueryClient、大小写转换、基础 UI 控件。 禁止放业务状态

路由用 TanStack Router 但**不用文件约定生成**，而是手动 import 后

`rootRoute.addChildren([...])`。手写更啰嗦，但路由树是显式的、可搜索的，也避免了约定式生成在重命名时的隐式破坏。

■ 两种状态，两套工具

TanStack Query · 服务端状态

默认 `staleTime=30s`、`retry=1`、`refetchOnWindowFocus=false`。每个 feature 定义 `xxxKeys / xxxApi / useXxx`，负责缓存与失效。

Zustand · 高频本地态

`immer + devtools + subscribeWithSelector`。`canvasStore`（图与 dirty 标记）、`agentCanvasStore`、`executionStore`（节点输出/重试/介入）、`authStore`、`agentConversationStore`（消息流/沙箱/终端）。

分界线：「能从服务器重新拉到」的用 Query，「毫秒级变化或正在编辑」的用 Zustand。画布自动保存就是 store subscribe + 2 秒防抖 PATCH。

■ ky 拦截链

- `beforeRequest` 注入 Bearer
- `beforeRetry` 仅对 401 **重试一次**：
`refreshSession()`，失败即 `signOut` 并跳 `/login`
- `afterResponse` 全局 `snake_case → camelCase`；写请求走 `toSnakeBody()`

只重试一次是为了避免刷新失败时的请求风暴，同时把「会话真的过期了」这件事尽快告诉用户。

■ 执行监控的两段式

`useExecutionSocket` 连接后发 `execution:subscribe {tenantId, executionId, lastEventId}`，ACK 回 `currentState`：先合并快照，再按 `eventId` 单调推进游标。重连 5s 起步、上限 30s、无限重试。

`useExecutionMonitor` 把回调桥进 `executionStore`；状态事件若带 `result / checkpointData` 会立即恢复一次性输出，避免用户必须刷新详情页才看到结果。

E2 · CANVAS CONTRACT

两套注册表

工作流画布 · NODE_TYPE_REGISTRY

```
llm-model · http-tool · code-tool · mcp-  
tool · sandbox · manual-trigger · schedule-  
trigger · webhook-trigger · api-event-  
trigger · knowledge-base · text · text-  
output · json-output · condition · loop ·  
iteration · loop-start · iteration-start ·  
loop-state · result · break · continue ·  
reusable-block · smart-routing · plugin ·  
input-preprocessor · memory · agent · skill  
· workspace · merge
```

Agent 配置画布 · AGENT_CANVAS_NODE_REGISTRY

```
agent-main · llm-model · smart-routing ·  
mcp-tool · knowledge-base · memory · text ·  
sub-agent · input-preprocessor · skill ·  
sandbox · workspace
```

这不是执行 DAG，而是一个用图的形式表达的参数编辑器。

agent-main 上挂 nativeToolPolicy 和
selfEvolutionPolicy。

GOTCHA

node.type 只代表渲染类别，真正的业务类型必须读 node.data.nodeType。这是历史包袱，也是新人最容易踩的坑。

■ 端口定义

```
PortDefinition: id / label / direction / dataType /  
required / multiple / maxConnections / schema /  
acceptsAnyDataType。schema 支持 Scalar / Object / Array。
```

边上另存一整套兼容性元数据：rawCompatibilityLevel、
visualLevel、reasonKey、missingFields、
candidateMappings、fieldMapping、mappingSummary、
transformFn。

这么重的边数据是为了让「这条连线为什么是黄色的、缺哪几个字段、可以
怎么映射」在重新打开画布时不用重算——它随图一起存进
workflow_definitions.edges，因此也会进入发布快照。

■ 两阶段校验

- 01 同步 guard。拖拽 hover 时只读缓存和
arePortDataTypesCompatible()：
exec/volume/memory 严格匹配，其余允
许 text↔json、json↔array、
skill↔text。runSynchronousGuards()
再查端口存在性、自环、compound 边界、
目标 maxConnections。
- 02 异步权威判定。onConnect 时调
evaluateConnection() →
TypeEngineService → WASM。

WHY TWO

React Flow 的连接回调是同步的，不能 await。所以先用便宜的规则挡掉明显错误保证拖拽跟手，再用贵的 schema 深比较给最终结论。

E3 · TYPE ENGINE

用 Rust 判断两个端口能不能连

■ 四级结论

LEVEL	含义
EXACT	完全一致，直连
TRANSFORM	需要一次确定性转换（给出 transformFn）
PARTIAL	结构部分匹配：给出 missingFields 与 candidateMappings
INCOMPATIBLE	拒绝，附 reasonKey 与 conflictPath

PARTIAL 允许连线——因为大多数真实场景就是「字段名不一样但语义相同」。二值拒绝会逼用户在中间插一堆转换节点。边上持久化候选映射，让用户在配置面板里确认。

导出三个 WASM 函数：checkCompatibility、checkSchemaCompatibility、validateSchema。

■ 运行时韧性

- Worker 里单例 WASM，加载优先 instantiateStreaming，失败回落 arrayBuffer
- 按稳定端口签名做 cache key；inFlight 合并重复请求
- 单次请求 4 秒超时
- Worker crash 或 init fatal：清空 ready / cache / inFlight 并 terminate()
- Service 捕获后走 fallback.ts ——受控降级，不是吞异常

Criterion benchmark 用 10 字段对象做基准，集成测试要求 < 100ms。

■ 为什么值得引入 Rust

- 规则单一来源。**同一套判定逻辑可以同时给浏览器和服务端用，不会出现「前端说能连、后端说不能」。
- 递归 schema 比较 + 字段相似度**是纯计算，放主线程会掉帧，放 Worker 天然合适；WASM 的性能让它在拖拽交互中感觉不到延迟。

（第二点在代码里没有写明唯一动机，属于架构推断。）

REALITY: DRIFT

这个「单一来源」目前**没有真正做到**。Rust src/types/port.rs 是 12 个 PortDataType；Studio 的 typeSchema.ts 已经扩到 14 个（多出 exec、volume）；文档里写的「canonical 10 值」是更早的历史描述。**四处镜像已经漂移**——见 F1。

数据库拖走也读不出

■ 混合加密

LlmEncryptionService：随机 32 字节 DEK + 12 字节 IV，AAD = tenantId:timestamp，AES-256-GCM 加密载荷，再用租户 **RSA-OAEP-4096 公钥**包住 DEK。返回 ciphertext / encryptedSessionKey / iv / authTag / aad / fingerprint / algorithm，finally 里把 DEK 清零。

数据库里只有公钥。私钥只存在租户浏览器的 IndexedDB（agentloom-keystore / private-keys，PKCS8 二进制），解密时用 importKey('pkcs8', ..., extractable:false) 导入不可导出的 CryptoKey。服务端**不持有该私钥，也没有对应的解密路径**——注意措辞：服务端另有一套自管 KEK 的信封解密（MCP 凭证、私有部署 secret 等），那是独立密钥体系，不能笼统说「服务端没有任何解密函数」。

但要把边界说准：**加密发生在写库之前，明文在 worker 进程里存在过**，模型提供商也看得到。所以它保护的是**静态数据**——拖库、备份泄露、DBA 越权读不到内容；它**不保护**「在线服务进程被攻陷」这个场景。把它讲成「服务端从未见过明文」是会被拆穿的。

加密对象	处理
LLM 输出	agent-task.worker.ts 把 result.content 换成 [ENCRYPTED]，密文入 encryptedContent
agent_decision / tool_output 证据	整个 packet 加密，库里存 encryptedPacket + 脱敏摘要，isEncrypted=true

■ 密钥表是 append-only

tenant_encryption_keys：organization_id + key_fingerprint 唯一，外加**单 active 的 partial unique index**。rotateKey 先把旧 active 置 rotating 再插新 active，失败会恢复旧状态。

为什么不原地更新公钥？因为**历史密文必须能被对应的历史私钥解开**。覆盖公钥等于把过去的**数据永久变成乱码**。

TRADE-OFF TO ADMIT

agent-task.worker.ts 在加密失败时会 **warn 后明文降级**。这是「可用性优先于机密性」的选择，面试时应该主动说出来——更严格的做法是加密失败即执行失败。

SUPABASE PKCE
授权码 + code_verifier，防授权码窃取。回调页
exchangeCodeForSession；
authStore 监听 SIGNED_IN / TOKEN_REFRESHED / SIGNED_OUT，从 JWT 顶层取 tenant_id，缺失则进

E4 onchain加密

MFA TOTP
enroll (QR/URI/secret) → challenge → verify → unenroll，AAL1/AAL2 分级。移动端是服务端 REST TOTP 的原生实现。

API KEY
al_ + 32 字节随机，库里只存 **SHA-256 hash** 与 token_prefix；明文只在创建响应里出现一次。每用户每租户上限 20 个。

E5 · AUDIT

只能追加的两张表

`audit_logs` 与 `audit_log_archives` 列结构完全相同: `id`, `tenant`, `actor`, `event/resource/execution`, `summary`, `before/after/metadata (JSONB)`, `createdAt`。两张表都用 `createAppendOnlyTenantPolicies` ——数据库层只授 **SELECT** 和 **INSERT**, 没有 **UPDATE** / **DELETE** 权限。也就是说, **应用使用的 `authenticated` 角色没有能力改写或删除审计记录**; 能绕过的只有直接持有更高权限的数据库账号 (迁移/DBA), 那已经属于另一个信任层级。

■ 归档链路

- 01 `AuditLogRetentionScheduler` 用 `BullMQ` `upsertJobScheduler` + 固定 ID 注册**单例**周期任务 (多副本部署下只会会有一个)
- 02 `Worker` 按 `retentionDays` 算 `cutoff`, 分租户批量选出热表旧行
- 03 `INSERT INTO archives ... ON CONFLICT DO NOTHING`
- 04 **回读校验 `archive` 里确实有这些 ID**
- 05 校验通过才从热表删除; 校验失败**不删**

这个「copy → verify → delete」顺序保证任何一步崩溃都不会丢数据, 最坏情况只是数据同时存在于两张表——而读侧本来就会去重。

■ 合并读取

`readMergedRecall` 并行查热表与归档表, 按 `id` 建 Map 去重 (**先放 `archive`, 再用 `hot` 覆盖**), 最后按 (`createdAt`, `id`) 升序返回。`findById` 先热后档; `findResourceSequence` 同样合并。

对调用方来说归档是完全透明的。

INTERFACES

`GET /audit-logs`、`/audit-logs/:id`、`/audit-logs/resources/:type/:id/sequence`, 运行时权限固定 `owner / admin`。

HONEST

`AuditLogService.record()` 是**规范**唯一写入口, 但 `resource-governance.service.ts` 里有一个 `recordAuditIndependently` 兜底: 在没有事务上下文时自己开事务直写 `audit_logs`。别把「唯一入口」说得太绝对。

E6 · GOVERNANCE

七个配额，两种拒绝

■ `tenant_quotas`

字段	默认
<code>apiRateLimitPerMinute</code>	100
<code>maxConcurrentExecutions</code>	—
<code>dailyExecutionLimit</code>	—
<code>dailyApiCallLimit</code>	—
<code>storageQuotaMb</code>	—
<code>maxSandboxCpuPercent</code>	—
<code>maxSandboxMemoryMb</code>	—

`execution_governance_controls` 另存暂停开关：`scope` (tenant / workflow) + `targetId` + `status` (active / paused) + `reason`, (`org`, `scope`, `target`) 唯一。

■ 两种 HTTP 语义

分钟级速率限制。带 `Retry-After` 和 `X-RateLimit-Limit/Remaining/Reset`。语义是「等一下再来」。

409
日配额、并发上限、治理暂停。RFC 7807 problem+json。语义是「**现在这个状态就是不行**」，重试无意义。

区分这两个很重要：客户端 SDK 会对 429 自动退避重试，对 409 直接报错给用户。混用会导致要么无脑重试打爆服务，要么用户看到莫名其妙的失败。

`CustomThrottlerGuard` 的 tracker 是 `tenant:{tenantId}` (未识别租户时回落 `apikey:prefix / jwt:sub / ip`)，计数存 Redis。所有阻断都写审计。

■ 监控仪表盘

GET `/organizations/:id/monitoring` (owner / admin)。三档窗口：**15m/5m 桶、1h/10m 桶、24h/4h 桶**。事务内聚合执行记录、Agent 执行摘要、四类告警通知、两类治理阻断审计事件，以及 BullMQ `agent-task` 队列的当前快照。

输出 `summary` (执行数、成功率、失败率、平均时长、队列深度、治理阻断数、活跃告警) + `trend` + `alerts` + `hotspots` + `riskSummary` + `deep links`。

DETAIL WORTH TELLING

趋势里的 `queueDepth` 是 `null` 而不是 0。队列深度只有「当前」快照，没有历史时序。**宁可返回 null 让图上断线，也不伪造一个看起来合理的历史值**——监控数据一旦开始编，整个面板就失去了决策价值。

E7 · KNOWLEDGE & TOOLS

检索链路与外部工具

■ 文档入库

01 解析 按 MIME 分发: PDF (pdf-parse) / DOCX (mammoth) / Markdown / 纯文本	02 分块 <code>KnowledgeNodeFactory</code> 按知识库配置: sentence / sentence_window / markdown	03 落库 替换式写入 <code>knowledge_nodes</code>	04 向量化 按配置生成 embedding	05 索引 Qdrant collection <code>knowledge_<kbId></code> >, Cosine, 失败清理已 写向量
---	--	---	--	---

■ 检索：四段可插拔

- 01 Query orchestration.** `none` 或 `HyDE`—先让 LLM 生成一个「假设答案」当查询，因为答案的向量通常比问题的向量更接近真实文档。
- 02 向量召回。**多 query variant 分别检索，`mergeNodeScores` 去重取高分。
- 03 阈值过滤**，然后 `rerank` (`none` 或 Cohere topN, 密钥经 `DecryptionBoundary` 取)。
- 04 映射输出:** `chunk / node / content / location / document / file metadata`。有 `evidenceContext` 时发 `RAG_RETRIEVED` 证据事件。

FALLBACK

Qdrant 集合缺失或异常时，回落到 `knowledge_nodes` 的词法 token LIKE 查询。质量下降但不至于「知识库整个不可用」。

■ MCP 三种 transport

`StdioClientTransport`
(`command/args/env`)、
`SSEClientTransport`、
`StreamableHTTPClientTransport`。超时: 连接
120s / list 60s / call 60s。

`mcp_server_configs` 的凭证用信封加密存
`encryptedData / DEK / iv / tag`;
`findConfig` 只返回 `credentialKeys` 不返回值。

TWO-PLACE FAIL-CLOSED

`no_sandbox` 禁 stdio 在**两处**拦截: `agent-definition.service.assertRuntimeModeConstraints` (Agent 发布时) 和 `workflow-version.service.assertNoSandboxWorkflowMcpConstraints` (workflow 发布时)。只拦一处的话，用户可以先建合法 Agent 再在工作流里改配置绕过。

E8 · MOBILE

观测与操作，不做编辑器

Flutter 3.41.2 + Riverpod + GoRouter + Dio。
StatefulShellRoute.indexedStack 五个分支：`/dashboard`、`/workflows`、`/agents`、`/resources`、`/settings`。执行详情、Agent 对话、OAuth 回调、MFA 在 Shell 之外。宽度 ≥ 1080 用 `NavigationRail`，否则 `NavigationBar`。

■ 移动端有 / 没有

范围	能力
有	列表浏览、参数化启动、执行监控、Agent 对话（含工具调用瀑布流与权限审批）、Memory / Skill / 工作区 / 沙箱 / 知识库 / MCP / 模型的轻管理、原生 MFA、FCM 深链、附件上传
没有	React Flow 画布与节点配置、版本发布、组织成员管理、治理与自主策略、私有部署运维、插件与开发者控制台、监控面板、证据导出、优化建议、触发器与 Block Library

REALTIME

原生端 transport 强制 `websocket`，Flutter Web 保留 `polling` → `websocket` 升级（适配代理）。事件同样是带 `eventId` 的 `ExecutionEventEnvelope<T>`（Freezed 生成）。断连后每 5 秒 `GET /executions/:id` 轮询兜底，重连再切回 WS——WS 优先、REST 保底。

■ 三个值得讲的细节

独立的 auth Dio

AuthApi 用一个不挂 `AuthInterceptor` 的 Dio 实例——否则刷新 token 的请求自己也会触发刷新，形成死循环。

队列化 401

`QueuedInterceptorsWrapper` 序列化并发 401: `token-expired` 刷新重试，`token-revoked/invalid/missing` 直接登出。`stale-token` 比较避免重复刷新。

Web-first onboarding

注册成功不保存 `session`，而是用 `url_launcher` 打开 `Web/login?returnUrl=/onboarding`。因为新用户还没有租户，存一个 `tenant_id=null` 的半初始化会话会污染后续所有请求。

E9 · DEPLOYMENT

一个入口，一个特权进程

■ Compose 拓扑

OpenResty 是唯一公网端口（默认 8080）：`/api/` 与 `/socket.io/` → `server:3000`（后者升级 WebSocket），`/auth/` → Supabase Kong，`/` → `studio:8080`，`/healthz` 本地 200。

服务清单：`reverse-proxy` · `studio` · `docs` · `server` · `worker` · `firecracker-runtime` · `postgres:16` · `redis:7` · `minio` · `qdrant v1.17` · `createbuckets`，另有 `tools/migration profile` 下的 `server-migrator` 与 `sandbox-cutover`。Supabase 栈通过外部 `supabase-shared` 网络接入。

SERVER VS WORKER

同一个镜像、同一个入口 `node dist/src/main.js`，都跑完整的 Nest + BullMQ processors。拆成两个部署单元的边界是**暴露面与回调地址**：`server` 面向公网，`worker` 只在内网做队列与 `callback relay`，各自配自己的 `APP_SANDBOX_CALLBACK_BASE_URL`，可以分别限额扩容。两者都不挂 Docker socket / KVM / NET_ADMIN。

■ Helm

`server / worker / studio` 是普通 Deployment + HPA；`firecracker-runtime` 是 `replicas=1` 的 **StatefulSet**：RWO state PVC、privileged、只读 `rootfs`、`hostPID`、`hostPath` 挂 `/dev/kvm` + TUN + cgroup、60 秒终止宽限、mTLS readiness，必须固定到预检过的 `x86_64` 节点。

关闭内置依赖时模板会 `fail()` 强制要求提供外部连接串——避免「以为连了外部 PG，其实连的是被删掉的内置实例」。

■ 备份必须两边都做

`backup-postgres.sh` 用 `pg_dump -Fc` 产出 `.dump / .sha256 / .meta`，并用 `pg_restore --list` 验证结构可读；`backup-minio.sh` 用 `mc mirror` 做桶快照。默认各保留 7 天，`systemd timer` 每小时 :05 / :20，RPO < 1 小时。

`restore.sh` 顺序：校验 SHA 与结构 → 停应用 → 重建库 + `pg_restore` → `mc mirror --overwrite --remove` 回灌 → 启动 → 检查 DB、`/api/v1/health`、`/healthz`。

只备数据库是不够的：元数据在 PG，文档 / 工作区 / 插件包在 MinIO，缺一边恢复出来就是断的。

■ Firecracker 产物链

`build-artifacts.sh` 按 `artifact-lock.json` 下载并 **SHA-256 校验** Firecracker/jailer、内核、BusyBox、Arch `rootfs` digest；构建 guest 侧 Fastify（`npm ci + typecheck + build`）与静态 Go `agentloom-guestd`；组装 `rootfs`、2GiB `ext4`、内核与 `initramfs`；验证 `vmlinix` ELF；最后生成含 **git commit** 的 manifest，runtime 启动时再校验一次。

WHY

沙箱镜像是安全边界的一部分。锁定 + 校验 + 启动时复验，保证跑的确是审过的那份产物。

PART F

F

复盘

主动交代技术债，比被追问出来强。这一部分先列出所有已知的实现缺口与契约漂移，再把最可能被问到的问题整理成可以直接说出口的答案。

F 1

技术债与契约漂移

F 2 - F 4

问答速查

F 5

自我介绍脚本

已知缺口清单

每一条都是实测发现的、当前代码与设计意图不一致的地方。被问到时应该直接承认并说出修复方向。

问题	现状	修复方向
端口类型四处漂移	Rust port.rs 12 值；Studio typeSchema.ts 14 值（多 exec/volume）；文档写 10 值	抽 @agentloom/contracts 单包，Rust 端由它生成；加跨包一致性测试
对话回放未接通	/agent-conversation 前端 store 有 lastEventId 字段，但 subscribe 不传、事件处理不更新游标	照抄 /execution 的两段式（snapshot + 增量）实现
入队不是事务性发件箱	afterCommit 解决了「先发后提交」，但提交后进程崩溃仍会丢任务	引入 outbox 表 + 轮询补偿
节点副作用非幂等	checkpoint 保证调度层可恢复，但 HTTP / 写文件类节点重跑会重复执行	节点级幂等键 + 结果去重表
加密失败明文降级	agent-task.worker.ts 加密异常时 warn 后写明文	改为按组织策略选择「失败即失败」还是「降级」
审计写入口非绝对唯一	resource-governance.service.ts 有独立直写 audit_logs 的兜底	把该路径也收敛进 AuditLogService
优化建议面板不可达	OptimizationSuggestionsPanel 无生产挂载点，只被测试引用	接进节点配置面板，或删掉避免僵尸代码
CLI 命令与文档不符	keys 只有 generate 没有 list；publish 不做真正上传	补齐实现或修正文档，二选一
兼容层未启用	createVercelStreamFn 写好但无生产调用点	接入或移除
文档英文版只有壳	zh/ 8 章 36 篇，en/ 只有首页 stub	要么补，要么先关闭 en locale
runtime manager 单点	replicas=1 + RWO + 绑节点，无 HA	需要 microVM 状态复制才谈得上 HA，属于明确的取舍而非疏忽
没有 CI/CD	只有本地测试与覆盖率门槛	最低限度接一条跑 lint + test + typecheck 的流水线

HOW TO SAY IT

面试里讲技术债的正确姿势是「问题 + 影响范围 + 我会怎么修 + 为什么当时没修」。最后一项尤其重要——「当时优先做了 X，因为 Y」比「没时间」有说服力得多。

F2 · Q&A · ARCHITECTURE

架构类追问

Q1 为什么 Agent 和 Workflow 是两个顶层概念，而不是一个？

生命周期和恢复边界不同。工作流执行是一次性 DAG，有确定终点，需要逐步重放与审计；Agent 对话长期存在、可跨会话继续、要独立版本与权限策略。合并会让其中一方被另一方的模型扭曲。用 `WorkflowAgentAdapter` 做协议桥接，保证 Agent 逻辑只有一份。

Q2 多租户为什么用 RLS 而不是在代码里加过滤条件？

代码过滤的正确性上限等于代码审查严格程度，一个新人漏写一次就是数据泄露。RLS 把隔离下沉到数据库：`FORCE ROW LEVEL SECURITY + SET LOCAL ROLE authenticated + 事务级 app.current_tenant`，即使拿到裸连接也查不到别的租户。代价是每个请求必须开事务，响应要等提交，副作用得注册 `afterCommit`。

Q3 调度器为什么每次都回数据库查前驱状态，不用内存计数器？

内存计数器在进程重启后就丢了，多 worker 并发推进同一张图时还会不同步，导致卡死或重复调度。数据库是唯一真相源，多几次查询换来重启可恢复。但别说成「水平扩展安全」：并发正确性另外依赖状态迁移上的比较并交换（`WHERE id=? AND status=<刚读到的值>`），而且竞争失败方是抛异常而非良性无操作，CAS 之前写入的步骤输入也不受保护，我没做过并发压测。

Q4 DAG 里怎么表达循环？

不画回边。循环体是带 `parentId` 的子图，拓扑排序时先 `filterTopLevel` 排除它们；每一轮开始前 `reset` 子步骤的 `input/result/checkpoint`。这样每个内部节点在 `execution_steps` 里始终只有一行，轮次信息记在父节点的 `rounds` 里。代价是历史轮次的中间产物只能从事件流回看。

Q5 为什么工作流执行钉死快照，Agent 对话却自动跟随新版本？

因为可复现性要求不同。工作流执行必须能被审计和重放，配置中途变形不可接受；对话本身就有模型随机性，不可能严格复现，让 bug 修复无法到达老对话反而更糟。同一个问题，两种实体给了相反答案，理由是需求不同而不是实现不一致。

Q6 没有共享类型包，跨端契约怎么保证？

靠约定手工镜像——而且已经漂移了：端口类型在 Rust 是 12 值、Studio 是 14 值。这是这个项目最明确的架构失误。正确做法是抽一个 `contracts` 包生成各端类型，并加跨包一致性测试。

Q7 为什么用非标准 monorepo？

各包独立 lockfile，让 SDK 能停在 Zod 3（插件生态兼容）而服务端用 Zod 4，Rust / Go / Flutter 也本来就无法进 `pnpm workspace`。代价是没有一键构建和统一依赖升级。

F3 · Q&A · CONCURRENCY

并发与一致性追问

Q8 入队为什么必须在事务提交之后？

经典的「消息先于数据可见」问题：worker 抢先按 `executionId` 查行，因为未提交 + RLS 而查不到，任务直接失败。所以入队注册成事务的 `afterCommit` 回调，并用 `jobId = executionId` 让 BullMQ 天然去重。还没解决的是「提交成功但入队前崩溃」——那需要 `outbox`。

Q9 两个人同时编辑一张图怎么办？

草稿表的 `version` 字段兼作 OCC 版本，每次保存 +1，写入时带上期望版本，不匹配就拒绝。优化建议 `apply` 时还叠了第二层 `guard`： `WHERE status='pending'`，防止同一条建议被消费两次。并发写必须同时防「覆盖别人」和「重复自己」。

Q10 断线重连怎么保证事件不丢也不重？

每个执行维护单调递增 `eventId` 和 500 条环形缓冲。客户端带 `lastEventId` 订阅：缓冲够就补增量；缓冲不够就发完整 `state snapshot` 让客户端整体覆盖；没落后就什么都不发。终态事件广播后要等 30 秒再清缓冲，给刚好错过的客户端留窗口。

Q11 事件太多把前端打爆怎么办？

令牌桶 + 每执行队列（cap 500, 100ms drain）。队列满时丢最旧的，因为 LLM 流式 token 越新越有用。终态不是靠「插队」保证的——是先 `flush` 排空该执行的队列，再广播终态。（标题事件属于 `/agent-conversation namespace`，不在 `/execution` 这套背压里。）

Q12 同一个工作区被两个沙箱挂载怎么办？

`workspace_runtime_leases`： `workspace_id` 唯一 + `fencing token` + 过期时间。挂载前 `acquire`，恢复后 `assertHeld`，最后一个 `mount` 离开才释放并回写快照。没有 `fencing token` 的锁是假锁——卡住的旧持有者恢复后仍会写入。

Q13 沙箱管理器重启后残留的虚拟机怎么处理？

`Recover()` 扫描： `persistent` 且磁盘 `e2fsck` 通过就标记 `stopped` 可重启；但 `jailer / cgroup / PID / netns` 清理只要有一项失败，就保留 `handle` 并置 `failed`、拒绝继续。宁可留个需要人工处理的资源，也不冒两个 VM 抢同一块可写磁盘的风险。

Q14 计费怎么保证不重复算钱？

两层幂等：worker 在租户事务内跳过 0 值与已存在记录，service 层再用 `onConflictDoNothing(plugin, period)` 兜底。钱相关的写入必须一层在业务逻辑、一层在数据库唯一约束。另外用量记录虽然是 `await` 的，但外面包着只 `warn` 的 `try/catch`——记账失败不会让用户的执行失败。

安全类追问

Q15 为什么用 Firecracker 而不是 Docker 跑 Agent 代码？

容器共享宿主内核，一个内核漏洞即逃逸。Firecracker 每个沙箱独立内核，逃逸要先打穿 KVM/virtio——攻击面从**整个系统调用接口**收窄到**虚拟设备接口与虚拟机监控器**，启动开销又远低于完整虚拟机。而且**业务进程根本不持有 KVM 和 Docker socket**，权限集中在一个单例 Go manager。

Q16 网络隔离具体防什么？只写 nftables 不行吗？

不行。guest 里的 root 可以改自己的 IP 和 MAC，按 IP 写的规则形同虚设。所以在 host 侧 veth 的 **tc ingress 同时校验源 IPv4 + MAC 以及 ARP 的 sender IP + MAC**，其余帧全丢；guest 内核禁 IPv6 堵旁路；nftables 再拒绝 sibling VM、private CIDR 和非 relay host；manager 连 guest 时按分配 IP 校验证书 IP SAN 且禁止 ServerName。把**网络地址变成不可伪造的身份，这是关键**。

Q17 沙箱里的工具要回调服务端，怎么不被利用成 SSRF？

guest 永远拿不到真实地址。manager 在代理请求时把回调 URL 改写成不透明 relay URL + 32 字节随机 token；回调进来时先校验 RemoteAddr 等于该 guest 的分配 IP，再校验 token，然后 manager 代持真实上游凭证转发到白名单主机（只有 server/worker）。**token 24 小时过期，注册表上限 10000 条**。

Q18 插件签名为什么不直接对 ZIP 文件签？

ZIP 字节表示不唯一——条目顺序、压缩级别、时间戳都会变，任何无害的重打包都会让签名失效。所以对 **canonical descriptor**（深度排序的 manifest + 按路径排序的 [path, sha256] 列表）签名，只绑定业务内容不绑定容器格式。用 RSA-PSS，验签侧 saltLength 用 AUTO。

Q19 号称端到端加密，服务端真的解不开吗？

要先把边界说清楚，否则这个说法会被拆穿。**准确的表述是「服务端没有持久化的解密能力」，而不是「服务端从未见过明文」**——LLM 输出是在 worker 进程里产生的，加密发生在写库之前，所以在线进程和模型提供商本来就在信任边界之内。它真正防的是**静态数据泄露**：数据库里只有 RSA-4096 公钥，私钥只在租户浏览器 IndexedDB 以 PKCS8 存放、导入时 extractable:false，**服务端不持有该私钥、也没有对应的解密路径**，所以拖库、备份泄露、DBA 越权都读不到这部分内容。**说话要精确**：服务端并非「没有任何解密函数」——它有一套自管 KEK 的信封解密，用于 MCP 凭证、私有部署配置这类服务端 secret；那是另一套密钥体系，与租户 E2EE 无关。密钥表 append-only（org+fingerprint 唯一 + 单 active partial index），轮换插新行而非覆盖，否则历史密文永久变乱码。**另外两个要主动承认的口子：IndexedDB 仍受同源脚本影响；加密失败时当前实现会 warn 后明文降级**。

Q20 审计日志怎么保证不可篡改？

数据库层只授 SELECT + INSERT，没有 UPDATE/DELETE 权限。归档走 **copy → 回读校验 → 才删除**，校验失败不删。读侧并行查热表与归档表，按 id 去重、按 (createdAt,id) 排序，对调用方完全透明。

Q21 让模型改自己的配置，风险怎么控？

工具拆成读 / 提案 / 写三类，只有 apply_change 和 create_resource 需要人工审批。审批状态是 **tool-level 的 awaiting_permission**，**step 保持 running**，因为等待的只是一次工具调用，模型这一轮还在进行中。审批记录存 Redis 跨进程共享，可按「会话 × 分类」记住决定。**服务端等待审批的超时是 300 秒（SELF_EVOLUTION_PERMISSION_TIMEOUT_MS）**；沙箱 guest 侧那条 permission 回调 HTTP 请求另有 30 秒 abort，两个数字别搞混。

F5 · SCRIPT

三个长度的说法

「AgentLoom 是一个多智能体工作流编排平台。用户在画布上把 AI Agent、工具、知识库、条件循环连成 DAG 并执行；Agent 也可以脱离工作流独立对话。技术上是 NestJS + PostgreSQL RLS 多租户、BullMQ 调度、Socket.IO 实时流，Agent 代码跑在 Firecracker 微虚拟机里，第三方插件跑在 Extism WASM 沙箱里。**整个系统的组织原则是：不可信代码只在受控边界内执行，业务进程本身不持有任何宿主级权限。**」

2 分钟版 · 加这三段

- **数据一致性怎么做的。**每个请求包在设置了 `app.current_tenant` 的事务里，RLS 兜底；副作用注册 `afterCommit`；写冲突用 OCC + 状态 guard 双层防护。
- **实时怎么做的。**带单调 `eventId` 的事件信封 + 500 条环形缓冲 + 断线按 `lastEventId` 增量回放，缓冲不够降级为整体快照；令牌桶背压，丢最旧保最新，终态前先整体 flush 排队列再广播。
- **隔离怎么做的。**特权收敛到单例 Go manager；网络层用 tc 做 IP+MAC+ARP 三重绑定把地址变成身份；回调经不透明 relay 中转防 SSRF。

如果被问「AI 写的还是你写的」

直接承认高强度使用 AI 辅助，然后**立刻把话题拉到判断力上**：

「AI 能写出代码，但决定不做什么是我的工作。比如：为什么工作流不自动重试而 `agent-task` 重试 4 次；为什么监控趋势里 `queueDepth` 返回 null 而不是 0；为什么冷恢复清理失败要 `fail closed` 留个脏资源给人工。**这些都是取舍，不是生成。**」

再补一句技术债清单（F1）——能准确说出自己系统的缺陷，是最有力的真实性证明。

■ 三个可以主动抛出的「亮点故事」

tc 层身份绑定

「按 IP 写防火墙规则是没用的，因为 `guest` 能改自己的 IP。」——这句话本身就能说明你真的想过威胁模型。

canonical archive 签名

「对 ZIP 签名是错的，因为容器格式不唯一。」——展示你懂得区分「内容」和「表示」。

tool-level 权限门

「step 保持 running 而不是 waiting，因为暂停的是一次工具调用不是整个回合。」——展示你理解 agent loop 的状态机。

先讲取舍， 再讲实现。

面试官判断的不是你写了多少代码，而是你能不能解释每一个决定背后的约束。这份文档里的每一节都为此准备。

生成方式

源码实测 + 七路并行调研，数字均来自本地工作树命令输出

视觉体系

瑞士国际主义 · IKB 克莱因蓝 · Inter / Noto Sans CJK / JetBrains Mono

仓库

interview-brief/ · `node build.mjs` 可重新生成